

[← Go Back](#)

Library

Recents viewed

[Arduino SigFox for MKRFox1200](#)[autowp-mcp2515](#)[Arduino_MKRENV](#)[Arduino_MKRGPS](#)[Arduino_APDS9960](#)[Arduino_MKRIoTCarrier](#)[Home](#) / [Programming](#) / [Library](#) / **Arduino SigFox for MKRFox1200**

ON THIS PAGE

[Usage/Examples](#)[Compatibility](#)[Releases](#)[Sigfox Class](#)

+

DEVICE CONTROL

Arduino SigFox for MKRFox1200

[ARDUINO](#) [LGPL-2.1](#) [V1.0.5](#) [Arduino](#) [07/11/2022](#)[Arduino LLC](#)<https://www.arduino.cc/en/Re...>

Helper library for MKR Fox 1200 board and ATAB8520E Sigfox module

This library allows some high level operations on Sigfox modules, to ease integration with existing projects

GO TO REPOSITORY

[Usage/Examples](#)[Compatibility](#)[Releases](#)

This library allows you to use the [ATMEL SigFox transceiver \(ATAB8520E\)](#) on the [Arduino MKR FOX 1200](#) board. You can find out more about this board through the following links:

[MKR FOX 1200 Store Page.](#)[MKR FOX 1200 Documentation Page.](#)

SigFox employs a cellular system that enables remote devices to connect using Ultra-Narrow Band (UNB) technology. It requires little energy, being termed Low-power Wide-area network (LPWAN).

Check the SigFox [coverage](#) in your area!

To use this library:

```
1 #include <SigFox.h>
```

Sigfox Class

[SigFox.begin\(\)](#)

Description

Initializes the Sigfox library and module

[Help](#)

Syntax

```
1 SigFox.begin();
```

Parameters

None

Returns

true if correctly configured, false otherwise

Example

```
1 #include <SigFox.h>
2 #include <ArduinoLowPower.h>
3
4 void setup() {
5   Serial.begin(115200);
6   while (!Serial) {};
7
8   if (!SigFox.begin()) {
9     Serial.println("Shield error");
10    return;
11  }
12
13 void loop(){
14 }
```

▼ SigFox.beginPacket()

Description

Begins the process of sending a packet

Syntax

```
1 SigFox.beginPacket();
```

Parameters

None

Example

```
1 #include <SigFox.h>
2 #include <ArduinoLowPower.h>
3
4 void setup() {
5   Serial.begin(115200);
6   while (!Serial) {};
7
8   if (!SigFox.begin()) {
9     Serial.println("Shield error");
10    return;
11  }
12
13  void loop(){
14    SigFox.begin();
15    SigFox.beginPacket();
16    SigFox.print("123456789012");
17    SigFox.endPacket();
18    while(1);
19  }
```

▼ SigFox.write()

Description

Sends binary data to the SigFox's backend. This data is sent as a byte or series of bytes; to send the characters representing the digits of a number use the `print()` function instead.

Syntax

```
1 SigFox.write(val)
2 SigFox.write(str)
3 SigFox.write(buf, len)
```

Parameters

`val`: a value to send as a single byte

`str`: a string to send as a series of bytes

`buf`: an array to send as a series of bytes

`len`: the length of the buffer

▼ SigFox.print()

Description

series of characters; to send the binary data use the `write()` function instead.

Syntax

```
1 SigFox.print(val)
2 SigFox.print(str)
3 SigFox.print(buf, len)
```

Parameters

`val`: a value to send characters representing the value

`str`: a string to send as a series of characters

`buf`: an array to send as a series of characters

`len`: the length of the buffer

▼ SigFox.endPacket()

Description

Called after writing SigFox data to the remote connection, completes the process of sending a packet started by `beginPacket`.

Syntax

```
1 SigFox.endPacket();
```

Return

Returns an int: 1 if the packet was sent successfully, 0 if there was an error

Example

```
1 #include <SigFox.h>
2 #include <ArduinoLowPower.h>
3
4 void setup() {
5   Serial.begin(115200);
6   while (!Serial) {};
7
8   if (!SigFox.begin()) {
9     Serial.println("Shield error");
10    return;
11  }
12
13  void loop() {
14    SigFox.begin();
15    SigFox.beginPacket();
16    SigFox.print("123456789012");
17    int ret = SigFox.endPacket();
18    if (ret == 0)
19      Serial.println("OK");
20    else
21      Serial.println("KO");
22    while (1);
23  }
```

▼ SigFox.parsePacket()

Description

Checks for the presence of a SigFox packet, and reports the size. parsePacket() must be called before reading the buffer with SigFox.read().

Syntax

```
1 SigFox.parsePacket()
```

Parameters

None

Returns

int: the size of a received SigFox packet

▼ SigFox.statusCode()

Description

Returns the protocol status code

Syntax

```
1 SigFox.statusCode(protocol);
```

Parameters

protocol: can be one of either:

SSM
ATMEL
SIGFOX

Returns

the status code of the chosen protocol:

SSM

TBD

Atmel

Bit0: PA on/off indication Bit6: System ready to operate (system ready event) Bit5: Frame sent (frame ready event) Bit4 to Bit1: Error code

0000: no error
0001: command error / not supported
0010: generic error
0011: frequency error
0100: usage error
0101: opening error
0110: closing error
0111: send error

SIGFOX

0x00	: No error
0x01	: Manufacturer error
0x02	: ID or key error
0x03	: State machine error
0x04	: Frame size error
0x05	: Manufacturer send error
0x06	: Get voltage/temperature error
0x07	: Close issues encountered
0x08	: API error indication

`0x0A`
: Error getting frequency
`0x0B`
: Error building frame
`0x0C`
: Error in delay routine
`0x0D`
: callback causes error
`0x0E`
: timing error
`0x0F`
: frequency error

▼ SigFox.AtmVersion()

Description

Returns the Atm version

Syntax

```
1 SigFox.AtmVersion();
```

Returns

a String of 2 bytes containing the Atm version

▼ SigFox.SigVersion()

Description

Returns the module's firmware version

Syntax

```
1 SigFox.SigVersion();
```

Returns

a String of 2 bytes containing the SigFox version

▼ SigFox.ID()

Description

Returns the module ID. When a module is manufactured, a unique SigFox ID is recorded in its permanent memory. It is very important to keep and store the ID tray carefully, as it will be useful to insure the tracability of these devices and to register them on a SigFox Network Operator (SNO).

Syntax

```
1 SigFox.ID();
```

Returns

A String that contains the 4 bytes ID.

▼ SigFox.PAC()

Description

Returns the module PAC. For each module, a PAC key is a secret key corresponding to the Sigfox ID. The PAC key will be useful to register a device on a SigFox Network Operator (SNO). As opposed to the SigFox ID, a PAC key is not transferable and must be re-generated if the module's ownership is changed.

Syntax

```
1 SigFox.PAC();
```

Returns

A String that contains the 16 bytes PAC.

▼ SigFox.reset()

Description

Syntax

```
1 SigFox.reset();
```

Returns

None

▼ SigFox.internalTemperature()

Description

Returns the internal temperature sensor reading

Syntax

```
1 SigFox.internalTemperature();
```

Returns

a float representing the reading

▼ SigFox.debug()

Description

Enable debugging. Enabling the debugging all the power saving features are disabled and the led indicated as signaling pin (LED_BUILTIN as default) is used during transmission and receive events.

Syntax

```
1 SigFox.debug();
```

Parameters

None

▼ **SigFox.noDebug()****Description**

Disable debugging. Disabling the debugging all the power saving features are enabled and the led indicated as signaling pin (LED_BUILTIN as default) is not used during transmission and receive events.

Syntax

```
1 SigFox.debug();
```

Parameters

None

▼ **SigFox.end()****Description**

De-initializes the Sigfox library and module

Syntax

```
1 SigFox.end();
```

Returns

None

Example

```
1 #include "SigFox.h"
2 #include "ArduinoLowPower.h"
3
4 void setup() {
5   Serial.begin(115200);
6   while (!Serial) {};
7
8   // Uncomment this line and c
9   //if (!SigFox.begin(SPI1, 30
10  if (!SigFox.begin()) {
11    Serial.println("Shield erro
12    return;
13  else
14    SigFox.end();
15
16 }
17
18 void loop(){
19 }
```

▼ peek()

Description

Returns the next byte (character) of incoming serial data without removing it from the internal buffer. That is, successive calls to peek() will return the same character, as will the next call to read(). peek() inherits from the Stream utility class.

Syntax

```
1 SigFox.peek()
```

Parameters

None

Returns

the first byte of incoming serial data available (or -1 if no data is available) - int

▼ available()

Description

Get the number of bytes (characters)

buffer (which holds 8 bytes). `available()` inherits from the Stream utility class.

Syntax

```
1 SigFox.available()
```

Parameters

none

Returns

the number of bytes available to read

▼ `read()`

Description

Reads incoming SigFox data. `read()` inherits from the Stream utility class.

Syntax

```
1 SigFox.read()
```

Parameters

None

Returns

the first byte of incoming SigFox data available (or -1 if no data is available) - int

Example

```
1  /*
2   SigFox First Configuration
3
4   This sketch demonstrates th
5   Since the board is designed
6
7   This example code is in the
8  */
9
10 #include <SigFox.h>
11 #include <ArduinoLowPower.h>
12
13 void setup() {
14   Serial.begin(9600);
15   while (!Serial) {};
16
17   // Uncomment this line and
18   //if (!SigFox.begin(SPI1, 3
19   if (!SigFox.begin()) {
20     Serial.println("Shield er
21     return;
22   }
23   // Enable debug led and dis
24   // Comment this line when s
25   SigFox.debug();
26
27   String version = SigFox.Sig
28   String ID = SigFox.ID();
29   String DAC = SigFox.DAC();
```

Was this article helpful?



Connect and Contribute

[Project Hub](#)[GitHub Repository](#)[Forum](#)[Product Compliance](#)[Help Center](#)[Trademarks & Licensing](#)